Answer the following questions:

1. State true or false. When a new process is scheduled for execution, the operating system performs a *context switch,* which involves storing the state of the running process so that it can be restored for resuming execution at a later point.

   Answer: True

2. State true or false. The heap segment of a program occupies a fixed space in memory as the program runs.

   Answer: False

3. State true or false. Ready queue is the set of all processes residing in main memory, ready and waiting to execute.

   Answer: True

4. State true or false. The `init` process, process, which has process ID 1, is the ancestor of user processes in a Linux system.

   Answer: True

5. Name the Linux command that is used to list the system calls that are invoked by a process.

   Answer: `strace`

6. List three scenarios when a process enters a waiting state.

   Answer:
   a) waiting for user or keyboard input
   b) waiting for disk I/O completion
   c) waiting for a child process (`wait()`)

7. True or False. After `fork()`, parent and child processes can directly modify the same global variable in memory.

   Answer: False

8. What value does `fork()` return in the parent and child processes?
   a) It returns 0 in both the parent and child processes
   b) It returns the child's process ID in the parent process, and 0 in the child process
   c) It returns the parent's process ID in the child process, and 0 in the parent process
   d) It returns the same process ID in both processes

   Answer: b)

9. A parent process forks a child, then immediately calls `wait()`. While the parent is blocked, the child calls `fork()` again. How many total processes exist at that moment?
   a) 2 (parent and first child)
   b) 3 (parent, first child, and grandchild)
   c) 1 (only the first child)
   d) This scenario is impossible due to process blocking

   Answer: b)

10. If a child process terminates using `exit(7)` and `WIFEXITED(status)` is true, what will `WEXITSTATUS(status)` return?
   a) 7
   b) 7 shifted left by 8 bits
   c) 0 if successful, non-zero otherwise
   d) The process ID of the child

   Answer: a)

11. A child process calls `execl("/bin/ls","ls", NULL)` but the file `/bin/ls` doesn't exist on the system. What will happen?
   a) The child process will continue executing the original program after `execl()` returns
   b) The parent process will also fail
   c) The system will crash
   d) The child process will automatically terminate immediately when `execl()` fails

   Answer: a)

12. Consider the following C program:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
    int x = 10;

    pid_t p = fork();

    if (p > 0)
        x = 20;
    else if (p == 0)
        x = 30;

    printf("%d ", x);
}
```

What will be printed?

   a) Only 20
   b) Only 30
   c) Both 20 and 30, order can change
   d) 20 then 30, guaranteed in that order

   Answer: c)

13. Consider the following C program segment:

```c
pid_t child = fork();
execl("/bin/echo", "echo", "hello", NULL);
```

What will this code do?

a) Create a child process, and both parent and child will execute `/bin/echo`
b) Create a child process, but only the child will execute `/bin/echo`
c) Only the parent will execute `/bin/echo`; the child is unaffected
d) `execl()` will fail because it cannot be called after `fork()`

Answer: a)

14. A child process exits with `exit(42)` and the parent calls `wait(&status)`. What value is in status?

a) 42
b) 42 shifted left by 8 bits with additional status info in lower bits
c) 0
d) Undefined; it depends on the compiler

Answer: b)

15. Assuming standard line-buffered terminal output, how many times will "Start" be printed by this C program?

```c
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("Start\n");
    fork();
    printf("Start\n");
    return 0;
}
```

a) 1 time
b) 2 times
c) 3 times
d) 4 times

Answer: c)

16. How many times will "X" be printed by this C program?

```c
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();
    fork();
    printf("X\n");
    return 0;}
```

a) 2 times
b) 3 times
c) 4 times
d) 5 times

Answer: c)

17. How many times will "A" be printed by this C program?

```c
#include <stdio.h>
#include <unistd.h>
int main() {
 fork() && fork();
  printf("A\n");
 return 0;
}
```

a) 2 times
b) 3 times
c) 4 times
d) 5 times

Answer: b)

18. How many times will "A" be printed by this C program?

```c
#include <stdio.h>
#include <unistd.h>
int main() {
 fork() || fork();
  printf("A\n");
 return 0;
}
```

a) 2 times
b) 3 times
c) 4 times
d) 5 times

Answer: b)

19. How many times will "A" and "B" be printed by this C program?

```c
#include <stdio.h>
#include <unistd.h>
int main() {
 if(fork() && fork()) printf("A"); else printf("B");

 return 0;
}
```

a) A and B each printed once
b) A is printed once; B is printed three times

c) A is printed once; B is printed two times
d) A and B each printed two times

Answer: c)

20. When a parent process exits while its child process is still running, what happens to the child process?
    a) The child process is immediately terminated
    b) The child process continues running and becomes an orphan process adopted by the `init` (or `systemd`) process
    c) The child process becomes a zombie process (a terminated process whose exit status has not yet been collected by its parent)
    d) The operating system suspends the child process until a new parent is created

    Answer: b)